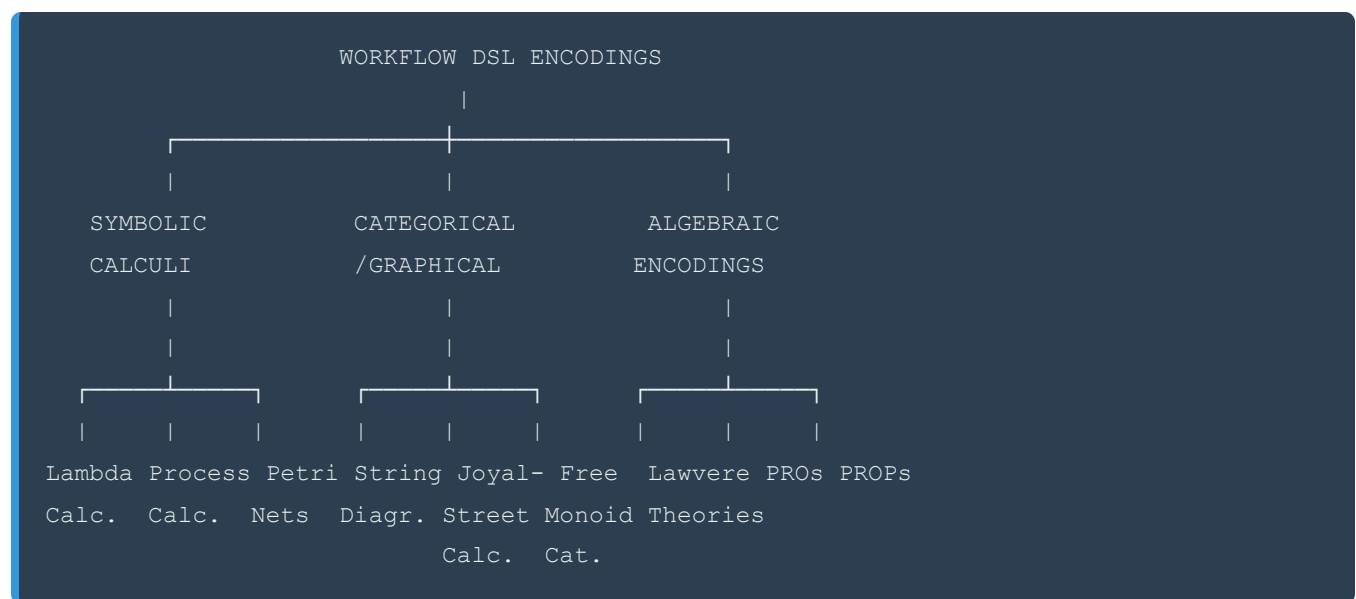


Formal Symbolic Encodings for Workflow DSLs - Visual Summary

Encoding Taxonomy



Compilation Pipeline



Key Theorems & Properties

Symbolic Calculi

- **Church-Rosser Theorem** (λ -calculus): Confluence guarantees deterministic reduction
- **Bisimulation** (Process calculi): Behavioral equivalence of concurrent systems
- **Soundness** (Workflow nets): Guarantee workflow completion and absence of deadlocks

Categorical Foundations

- **Mac Lane Coherence:** All canonical isomorphisms commute automatically - **Joyal-Street**

Soundness: Diagram deformation preserves morphism equality - **Completeness:** All derivable equations provable by diagram manipulation

Compilation Properties

- **Confluence**: Different optimization paths yield same result
- **Termination**: Rewriting always reaches normal form
- **Compositionality**: Semantics of whole = composition of parts

Real-World Implementation Matrix

System	Paradigm	IR	Optimization	Backend						
System 1	Paradigm 1	IR 1	Optimization 1	Backend 1						
System 2	Paradigm 2	IR 2	Optimization 2	Backend 2						
System 3	Paradigm 3	IR 3	Optimization 3	Backend 3						
System 4	Paradigm 4	IR 4	Optimization 4	Backend 4						
System 5	Paradigm 5	IR 5	Optimization 5	Backend 5						
System 6	Paradigm 6	IR 6	Optimization 6	Backend 6						
System 7	Paradigm 7	IR 7	Optimization 7	Backend 7						
System 8	Paradigm 8	IR 8	Optimization 8	Backend 8						
System 9	Paradigm 9	IR 9	Optimization 9	Backend 9						
System 10	Paradigm 10	IR 10	Optimization 10	Backend 10						

Apache Beam | Fluent API | Pipeline DAG | Fusion, Combiner | Multi-runner | | **TensorFlow** | Graph Builder | Comp. Graph | Grappler (DPO) | Session | | **Dask** | Lazy Eval | Task Graph | Cull/Fuse/Inline | Scheduler | | **Airflow** | Python DAG | Operator Graph | None (explicit) | Executor |

Formal Correspondences

Curry-Howard-Lambek Correspondence:

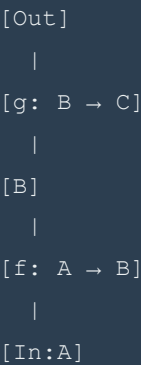
Logic		Type Theory		Category Theory
Proposition	$\leftrightarrow$	Type	$\leftrightarrow$	Object
Proof	$\leftrightarrow$	Term	$\leftrightarrow$	Morphism
Implication	$\leftrightarrow$	Function	$\leftrightarrow$	Exponential
Conjunction	$\leftrightarrow$	Product	$\leftrightarrow$	Cartesian Product
Workflow AND	$\leftrightarrow$	Pair	$\leftrightarrow$	Tensor Product $\otimes$

Computational Interpretations:

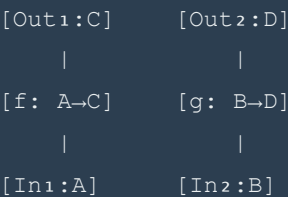
Category	Workflow Meaning
Object	Data type / Resource
Morphism $f: A \rightarrow B$	Task transforming A to B
Composition $g \circ f$	Sequential execution
Tensor $f \otimes g$	Parallel execution
Unit I	Empty workflow
Monoidal Category	Workflow algebra

String Diagram Notation Reference

Sequential Composition ($g \circ f$):



Parallel Composition ($f \otimes g$):



Symmetry (Swap):



Fork (Duplicate):





Join (Merge) :



Optimization Techniques Comparison

Technique	Representation	Search Strategy	Guarantees
Term Rewriting	Tree/DAG	Rule application	Confluence
DPO Rewriting	Category	Pattern matching	Compositionality
E-Graph	Equivalence graph	Saturation + Extract	Optimality
String Diagram	Graphical	Deformation	Coherence

Research Timeline

1930s: Lambda Calculus (Church)
1960s: Petri Nets (Petri)
1970s: Categorical Semantics (Lawvere, Mac Lane)
1980s: Process Calculi (Milner - CCS, π -calculus)
1990s: String Diagrams (Joyal-Street)
2000s: Arrows, Monadic DSLs (Hughes, Moggi)
2010s: E-Graphs, Equality Saturation
2020s: Compositional Workflow Systems (Beam, TensorFlow, Dask)

Key Papers by Topic

Foundational Theory

1. **Joyal & Street (1991)** - "Geometry of Tensor Calculus I" 2. **Mac Lane (1971)** - "Categories for the Working Mathematician" 3. **Selinger (2010)** - "Survey of Graphical Languages"

Compilation & Optimization

4. **Willsey et al. (2021)** - "egg: Fast Equality Saturation" 5. **Ehrig et al. (2006)** - "Algebraic Graph Transformation" 6. **Lafont (2003)** - "Algebraic Theory of Boolean Circuits"

Real-World Systems

7. **Akida et al. (2015)** - "Dataflow Model" (Apache Beam) 8. **Abadi et al. (2016)** - "TensorFlow: Large-Scale ML" 9. **Rocklin (2015)** - "Dask: Parallel Computation"

Process Calculi & Workflows

10. **Van der Aalst (1998)** - "Petri Nets to Workflow Management" 11. **Milner (1980)** - "Calculus of Communicating Systems"

Quick Reference: When to Use Which Encoding

Use Lambda Calculus When:

- ✓ Workflow is purely functional (no side effects) - ✓ Sequential composition dominates - ✓ Higher-order composition needed - ✗ Need explicit parallelism - ✗ Need resource tracking

Use Process Calculi When:

- ✓ Concurrent, communicating tasks - ✓ Mobile/dynamic workflow structure - ✓ Bisimulation-based equivalence needed - ✗ Need visual representation - ✗ State explosion is concern

Use Petri Nets When:

- ✓ Resource/token tracking essential - ✓ State-based workflow model - ✓ Need decidable verification (bounded nets) - ✓ Visual representation important - ✗ Need data transformation (use Colored Petri Nets)

Use String Diagrams When:

- ✓ Workflow has parallel composition - ✓ Visual reasoning important - ✓ Categorical semantics needed - ✓ Multi-input/multi-output operations - ✗ Need Turing-completeness

Use PROPs When:

- ✓ Need algebraic presentation - ✓ Multi-arity operations (m inputs, n outputs) - ✓ Symmetric monoidal structure - ✓ Generators + relations style - ✗ Need cartesian (copying/deletion) - use Lawvere theories

Use Monadic DSLs When:

- ✓ Embedded in functional language - ✓ Sequential effects dominant - ✓ Do-notation ergonomics - ✗ Need static analysis before execution

Use Arrow DSLs When:

- ✓ Need static structure analysis - ✓ Dataflow/stream processing - ✓ Want optimization before execution - ✓ Both sequential and parallel composition

Complexity Analysis

| Approach | Space | Time (Compile) | Expressiveness | Verification | |-----|-----|-----|
-|-----|-----| | **Lambda Calculus** | $O(n)$ | $O(n^2)$ CSE | Turing-complete |
Undecidable | | **Process Calculi** | Exp. (states) | Exp. (reachability) | Turing-complete |
Undecidable (general) | | **Petri Nets** | $O(n)$ | Poly (bounded) | Sub-Turing | Decidable
(bounded) | | **String Diagrams** | $O(n)$ | $O(n \log n)$ | Not Turing-complete | Decidable (equality) |
| **E-Graphs** | $O(n^2)$ worst | $O(n^3)$ saturation | Depends on rules | Optimal (extraction) | --- **Last**
Updated: October 19, 2025 **See Also:** FORMAL-SYMBOLIC-ENCODINGS-WORKFLOW-
DSLS.md (comprehensive analysis)